

程序员专属AI全栈系统化学习计划（2026完整版，全覆盖prompt/Agent/RAG/Vibe Coding/MCP等全技术栈）

一、计划总览

1. 适用人群

在职软件开发工程师，具备基础编程能力（前后端均可），**想要把AI融入日常开发、掌握AI编码工具、自研AI应用与智能体、落地企业级LLM工程方案**，拒绝碎片化学习，打通从提示词到AI原生架构的全链路能力。

2. 全覆盖核心技术清单

基础层：LLM大模型基础、Prompt提示词工程、Rule规则约束、Skills技能封装

应用层：Vibe Coding自然语言编程、AI编码工具（Cursor/Trae/CLAUDE Code/Codex）

架构层：RAG检索增强生成、AI Agent智能体、MCP多智能体通信协议

平台层：低代码AI平台（Coze扣子、Dify）；新兴开源Agent框架：OpenClaw、Hermes Agent

补充刚需必学技术（原有清单未覆盖，生产环境必备）：Ollama本地私有化部署、LLMOps大模型运维、Agentic RAG智能检索、KV缓存&上下文优化、Function Calling函数调用、AI缓存机制、大模型安全与代码审计、多模态能力基础

3. 周期与每日投入

- 总周期：8周（循序渐进，不占用整块工作日，适配程序员下班/周末碎片化学习）
- 每日投入：工作日1.5h，周末4h
- 核心目标：第4周实现AI提效日常开发，第8周独立搭建可商用的RAG+Agent+MCP一体化AI开发助手

4. 学习核心原则（Rule全局学习准则）

- 1. 代码优先原则**：所有AI学习必须落地可运行代码/可复用工程资产，拒绝纯理论空谈
- 2. 迭代复用原则**：每阶段产出的Prompt、Skill、Agent、RAG知识库全部沉淀为个人资产，后续项目直接复用
- 3. 边界约束原则**：所有大模型调用必须配置Rule规则，限制输出格式、代码规范、上下文长度、幻觉规避策略

二、分阶段详细学习路线（8周完整版）

第一阶段：筑基期（第1周）——LLM底层认知+Prompt工程+Rule规则体系

本周核心目标

搞懂大模型运行逻辑，精通程序员专属Prompt写法，掌握Rule规则约束，解决AI代码幻觉、输出不规范、答非所问问题，夯实所有AI技术的底层基础。

1. 核心知识点拆解

- **LLM大模型基础**：Transformer极简原理、上下文窗口、Token机制、模型幻觉成因、开源/闭源模型选型（适配代码场景：Claude3、GPT-4o、DeepSeek、Qwen）
- **Prompt提示词工程（程序员专项）**：零样本/少样本提示、CoT思维链、Few-shot代码示例、角色定制提示、结构化输出Prompt、代码Debug专属Prompt、需求拆解Prompt
- **Rule规则体系**：系统级硬规则、会话级软规则、代码输出强制规则、安全风控规则、Agent运行约束规则，学会用Rule根治AI自由发挥
- **Skills技能封装**：将高频开发能力封装为可复用AI技能（接口生成、SQL编写、代码重构、单元测试生成、注释自动补全）

2. 每日学习任务

天数	学习内容	实战产出
Day1-2	LLM基础理论、Token与上下文原理、代码场景模型选型	输出《程序员大模型选型手册》，区分编码、问答、长文本场景模型
Day3-4	通用Prompt框架（CRISPE/ROLE）、代码专用Prompt模板	沉淀10套可直接复用的开发Prompt：接口开发、Bug修复、代码优化、CRUD生成
Day5	Rule规则编写：强制代码规范、禁止冗余解释、指定代码版本、报错精准定位	编写通用全局Rule，后续所有AI调用默认挂载该规则
Day6-7	Skills封装：把高频开发动作封装为标准化AI技能包	搭建个人AI技能库，包含8个开发常用可调Skill

3. 本周必做实战

编写一套带Rule约束的完整代码生成Prompt：要求AI只输出可直接运行代码、附带极简注释、遵循团队ESLint规范、禁止多余话术、自动识别编程语言，彻底解决AI代码杂乱问题。

第二阶段：工具实操期（第2周）——AI编码工具全栈掌握+Vibe Coding

本周核心目标

吃透当下主流AI代码助手：Cursor、Trae、Claude Code、Codex，理解Vibe Coding新型编程范式，实现自然语言一键开发完整项目，告别逐行手写代码。

1. 核心知识点拆解

- **Vibe Coding（氛围编程）**：核心理念（人做架构决策，AI做代码落地）、和传统编码的区别、长会话连续开发、项目全局理解、多文件联动开发、需求模糊场景下的AI迭代开发
- **四大AI编码工具差异化精通**
 - Cursor：本地IDE无缝集成、Tab补全、全局代码索引、Agent代码助手
 - Trae：字节AI编码工具、实时代码预览、一键重构、免费大模型能力、国内网络零延迟
 - Claude Code：命令行AI编码工具、超长上下文、百万级Token读取整个项目、适合大型存量项目重构
 - Codex：OpenAI原生代码模型、底层能力理解、GPT系列代码生成底层原理
- **工具联动**：Cursor+Claude Code组合使用，实现IDE编码+命令行批量重构双场景提效

2. 每日学习任务

天数	学习内容	实战产出
Day1-2	Cursor全功能实操：代码对话、全局搜索、Agent模式、自定义Prompt与Rule	配置Cursor专属工作区规则，适配个人技术栈（Java/Go/Vue等）
Day3-4	Trae+Claude Code实操：Trae可视化开发、Claude Code命令行批量重构存量项目	用Claude Code一次性重构一个老旧后端项目，修复全量代码异味
Day5		

	Codex底层原理学习，理解AI代码生成的底层逻辑	手写简单API调用，直接调用Codex生成代码片段
Day6-7	Vibe Coding完整流程：需求描述→架构设计→AI自动编码→迭代调试→上线	纯自然语言无手写代码，完成一个完整前后端小项目（Todo管理系统）

3. 本周避坑Rule

Vibe Coding禁止完全放任AI自主开发，必须**架构层人工把控，代码层AI落地**；每完成一个模块人工审核，避免AI整体架构设计缺陷。

第三阶段：低代码AI平台实战（第3周）——Coze + Dify 快速搭建AI应用

本周核心目标

无需从零开发代码，利用低代码AI平台快速搭建对话机器人、知识库问答、自动化工作流，快速落地业务AI工具，理解Agent可视化编排逻辑。

1. 核心知识点拆解

- **Coze（扣子）**：字节低代码AI平台、可视化Agent编排、插件市场、一键发布网页/小程序/API、简单工作流、多模型切换
- **Dify**：开源可私有化部署AI平台、自定义RAG、灵活工作流、API对接、更适合企业私有化场景、代码自由度高于Coze
- 平台对比：Coze适合快速原型、对外分发；Dify适合企业私有部署、深度定制、对接内部业务系统
- 技能联动：将第一阶段沉淀的Prompt、Rule、Skills直接导入两大平台复用

2. 本周实战项目

1. Coze：搭建个人开发助手Agent，集成代码解释、SQL生成、接口调试三大技能，一键发布API供本地项目调用
2. Dify：私有化部署Dify，搭建团队内部代码知识库问答机器人，上传团队开发文档、接口规范

第四阶段：核心技术攻坚（第4-5周）——RAG检索增强生成 + 原生AI Agent开发

本周核心目标

解决大模型知识滞后、私有数据无法读取的痛点；理解Agent自主规划、工具调用、记忆能力，从低代码平台进阶到代码级自研RAG与Agent。

1. 第4周：RAG全链路学习

- RAG核心流程：文档切片→Embedding向量化→向量数据库存储→相似度检索→上下文拼接→LLM生成回答
- 关键优化点：切片策略优化、重排序Rerank、混合检索、幻觉抑制、代码专属RAG优化
- 工程落地：Chroma/Pinecone向量库、LangChain框架、私有代码知识库搭建
- 实战：搭建个人私有代码RAG，上传过往项目代码，实现代码复盘、旧项目逻辑问答

2. 第5周：AI Agent原生开发

- Agent核心模块：感知模块、规划模块、工具调用模块、短期/长期记忆模块、反射机制
- Agent运行Rule：自任务拆分规则、工具调用次数限制、失败重试规则、上下文清理规则
- Skill接入：将代码执行、接口请求、文件读写、RAG检索封装为Agent可调用Skill
- 实战：自研代码助手Agent，可自主完成：需求分析→代码生成→单元测试→代码优化→文档输出全流程自动化

第五阶段：高阶架构能力（第6周）——MCP多智能体通信协议

本周核心目标

掌握2026年AI工程最新标准MCP（Multi-Agent Communication Protocol）多智能体通信协议，实现多个专业Agent分工协作，解决单Agent能力局限问题。

1. 核心知识点拆解

- MCP定义：多Agent统一通信标准、消息格式规范、状态同步、任务分发、结果汇总
- 多Agent分工设计：需求拆解Agent、编码Agent、测试Agent、文档Agent、RAG知识库Agent
- MCP核心Rule：消息超时规则、权限隔离规则、任务冲突规避规则、结果校验规则
- 技术落地：基于LangGraph实现MCP协议适配，搭建多Agent协作集群；联动两大主流开源Agent框架做对比落地

- **OpenClaw（龙虾Agent）专项学习：**2026爆火TS开源自主Agent，轻量化本地部署，主打真实世界自动化操作，内置海量可复用Skills，支持浏览器自动化、文件批量处理、跨聊天网关对接；适配程序员日常批量代码整理、网页接口爬取、文档自动归档场景；优势是开箱即用、无需复杂代码开发，适合快速搭建轻量化自动化开发助手
- **Hermes Agent专项学习：**Nous Research开源自进化智能体，核心亮点为**自主进化技能、长期跨会话记忆、无人工干预自动迭代Skill**，突破传统Agent需要手动编写技能的痛点；支持本地私有化部署、兼容全系列LLM，适合打造长期值守、越用越智能的专属开发数字员工，可自动复盘编码历史、优化历史Prompt与工具调用逻辑
- **三大Agent横向对比：**自研原生Agent（灵活度最高、开发成本高）、OpenClaw（轻量化自动化、开箱即用）、Hermes（自进化长期记忆、无人值守）
- **新增刚需补充知识点（本周同步落地，贴合Agent&MCP场景）Function Calling 函数调用：**所有Agent运行底层核心能力，你原有计划未单独覆盖，是Agent调用工具、执行代码、对接外部接口的基础；区分硬编码工具调用+LLM原生函数调用，解决Agent不会精准触发技能、盲目调用工具的核心痛点，是OpenClaw/Hermes/MCP多智能体的底层基石
- **Agentic RAG 智能检索：**传统RAG是静态检索，Agentic RAG让Agent自主判断是否需要检索、拆分检索问题、迭代检索内容，解决传统RAG问答不准、代码知识库检索失效问题，适配代码类私有知识库场景
- **Ollama 本地大模型部署：**免费本地私有化LLM运行工具，无需外网、无Token费用，可本地跑代码专用大模型；适配Cursor/Trae本地离线编码、本地Agent调试，程序员日常开发刚需，彻底摆脱第三方模型接口费用与网络限制
- **KV缓存&上下文窗口调优：**解决长代码项目、多轮对话下LLM响应慢、Token消耗过高、会话卡顿问题，适配Claude Code百万级长上下文、Vibe Coding多文件连续开发场景

2. 本周实战

搭建5角色AI多智能体开发团队：通过MCP协议互通，输入一句业务需求，多Agent自动分工完成全流程开发；额外增加2组对比实战：1）基于OpenClaw快速搭建轻量化自动化代码运维Agent；2）基于Hermes部署自进化编码助手，让Agent自主优化代码生成Prompt与工具调用策略，对比三类Agent架构的落地差异与适用场景。

第六阶段：全栈整合与工程化落地（第7-8周）——全技术栈融合项目

本周核心目标

整合全部所学技术：LLM+Prompt+Rule+Skills+Vibe Coding+Cursor/Trae/CLAUDE Code+RAG+Agent+MCP+Coze/Dify，打造一款可商用、可私有化部署的程序员专属AI开发工作台。

项目整体架构（全覆盖所有关键词）

- 1. 底层底座：LLM大模型 + Codex代码模型 + Ollama本地私有模型
- 2. 约束层：全局Rule规则 + 会话级Prompt模板库 + 可复用Skills技能包 + AI缓存策略
- 3. 编码层：Cursor/Trae/CLAUDE Code本地工具联动 + Vibe Coding自然语言开发模式 + KV缓存长会话优化
- 4. 知识层：私有RAG知识库（代码文档+业务文档） + Agentic RAG智能迭代检索
- 5. 智能体层：单AI Agent核心调度 + Function Calling原生工具调用 + MCP多智能体协作
- 6. 应用层：Dify私有化后台 + Coze对外快速分发入口 + OpenClaw轻量化自动化节点 + Hermes长期值守自进化节点
- 7. 工程运维层（新增）：LLMOps模型监控、Token成本管控、AI代码安全审计、流式推理部署

第7周：功能开发与联调

完成工作台核心功能：代码一键生成、存量项目智能重构、私有知识库问答、多Agent自动化开发、全流程规则约束防幻觉；接入OpenClaw做外部自动化能力补充（网页接口测试、批量代码导出），接入Hermes做后台长期记忆与技能自动优化，实现工作台自我迭代升级。

第8周：优化、复盘、资产沉淀

- 性能优化：RAG检索速度优化、Agent响应耗时优化、MCP通信延迟优化
- 资产沉淀：整理全套个人AI资产库（Prompt模板、Rule规则、Skills封装体、Agent配置、RAG脚本、MCP协议代码）
- 职场落地：把AI能力融入日常工作，形成标准化AI开发工作流，提升日常编码效率60%以上

三、个人通用Rule规则模板（直接复制可用）



全局AI调用强制规则（挂载所有LLM会话）

- 1. 身份定位：你是资深软件开发工程师，输出内容贴合工业级开发标准；
- 2. 输出约束：代码优先，禁止冗余解释，所有代码必须可直接运行，标注依赖版本；
- 3. 格式约束：严格使用markdown格式，代码块标明编程语言；
- 4. 幻觉约束：不确定的内容直接告知，禁止编造接口、参数、文档；
- 5. 会话约束：自动记忆上下文开发需求，无需用户重复描述项目背景；
- 6. 工具约束：Agent调用工具单次不超过3次，复杂任务自动拆分分步执行。

四、每周复盘指标（自我考核标准）

- 第1周：能否零调试，用Prompt+Rule生成符合团队规范的工业级代码
- 第2周：能否纯Vibe Coding完成完整项目，熟练切换三大AI编码工具
- 第3周：能否1小时内基于Coze/Dify搭建可用业务AI工具
- 第4-5周：能否独立手写RAG与Agent核心代码，不依赖封装框架
- 第6周：能否理解并调试MCP多智能体通信问题
- 第1周：能否零调试，用Prompt+Rule生成符合团队规范的工业级代码
- 第2周：能否纯Vibe Coding完成完整项目，熟练切换三大AI编码工具，掌握KV缓存优化长会话卡顿问题
- 第3周：能否1小时内基于Coze/Dify搭建可用业务AI工具，理解平台内置Function Calling逻辑
- 第4-5周：能否独立手写RAG与Agent核心代码，掌握Agentic RAG优化方案
- 第6周：能否理解并调试MCP多智能体通信、Function Calling跨Agent工具调用问题，熟练使用Ollama本地私有化模型
- 第7-8周：能否整合全技术栈，落地LLMOps运维监控，产出完整可上线的AI工程化项目

五、学习避坑清单（程序员专属）

1. 不要沉迷纯Prompt玩法，必须落地代码工程，贴合开发本职工作
2. 不要忽视Rule规则，无约束的LLM和Agent无法用于生产环境，OpenClaw/Hermes同样需要自定义运行规则规避自动化误操作
3. Vibe Coding不是完全甩手掌柜，架构设计永远需要人工把控
4. RAG不要只做基础版本，必须掌握Agentic RAG智能检索优化，解决代码知识库检索失灵问题
5. MCP是未来Agent生态标准，务必提前布局学习，贴合AI工程就业趋势
6. OpenClaw适合短期高频自动化任务，Hermes适合长期无人值守智能运维，二者不可混用，需要根据业务场景选择对应Agent架构
7. Hermes自进化能力需要搭配严格Rule风控，避免自主迭代出不符合团队规范的代码与技能
8. **补充新增技术专属避坑点** Function Calling不要过度封装工具，避免Agent调用链路过长导致响应超时
9. Ollama本地模型无需盲目部署超大参数量模型，代码场景7B/14B模型性价比最优
10. 必须做AI代码安全审计，拦截AI生成的漏洞代码、硬编码密钥等安全风险
11. 生产环境务必接入LLMOps监控，管控Token开销、接口报错率、模型幻觉率

（注：文档部分内容可能由 AI 生成）